

Step 3 — Multiple MCP Servers at Once, With Tool Namespacing — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints. Each has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real routing structure), **Advanced** (keeping N connections open and closed correctly, which is a genuinely new problem Step 2 never had). Read Basic first — the core idea here (a lookup table) is simple; the depth is mostly in doing it for a variable number of servers cleanly.

- [Hint 1 — Why `call\_tool` needs a lookup table, and the exact separator gotcha](#)
- [Hint 2 — Connecting to N servers at once, and almost the whole pool class](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Why `call_tool` needs a lookup table, and the exact separator gotcha

Basic Version

Once there's more than one server, the model only ever sees *one flat list* of tool names — it has no idea some came from a "notes" server and some from a "filesystem" server, and it never needs to. What it does need is for every name in that list to be unique, so it can't accidentally ask for a `search` that could mean either one. Your code, underneath, needs the opposite: given the name the model picked, know exactly which server to actually call.

That's two small, related things: a way to make every name unique before the model ever sees it, and a way to map a unique name back to "this server, this original tool."

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Build the namespaced name as `f"{server_name}__{tool.name}"` while you're discovering each server's tools, and build a dictionary at the same time mapping that namespaced name back to the pair you'll need at call time:

```
routing = {} # namespaced_name -> (server_name, original_tool_name)
routing[f"{server_name}__{tool.name}"] = (server_name, tool.name)
```

At call time, the model hands you back exactly the namespaced name it was given (`tool_call.function.name`), so the lookup is a single dict access: `server_name, original_name = routing[namespaced_name]`. Then dispatch to whichever `ClientSession` belongs to `server_name`, calling it with `original_name` — the server itself never hears the namespaced name; namespacing exists purely for the model's benefit, one layer up.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the gotcha the README already warned you about, worth understanding *why* it's true, not just following it: OpenAI's function-calling `name` field is restricted to letters, digits, underscores, and hyphens — no `.`, no spaces. If you picture the natural, human-readable namespaced name as `notes.search_notes`, that dot will make the API call fail outright the moment you actually send it. `__` (double underscore) is a safe, valid substitute that still reads clearly as "this is a namespace separator," and — importantly — is very unlikely to collide with a real tool name a server author would actually pick (a single underscore is far more likely to appear inside an ordinary tool name, like `search_notes` itself, which is exactly why a *single* `_` would be a worse separator choice here: splitting `notes_search_notes` back into `( "notes", "search_notes" )` unambiguously requires knowing where the server name ends, but splitting on the *first* `_` would wrongly produce `( "notes", "search" )` if the tool itself were named `search_notes`).

That's also why you build the routing dict directly, instead of trying to split the namespaced string back apart at call time — string-splitting a name you generated yourself is solving a problem you don't have; you already know the mapping the moment you generate it, so store it and look it up.

One more edge case worth handling now: what if two *different* servers in your config file were accidentally given the same `server_name` ? Your routing dict would silently let the second one overwrite the first's tools in the lookup table. Decide what should happen (fail loudly at config-load time is the safer choice) before Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the two things you need (unique names for the model, a way back to the real server) without code. Intermediate gives the actual dict-based routing, built once at discovery time and looked up (not re-derived) at call time. Advanced explains *why* `__` is the right separator instead of a `.` or a single `_`, and adds the configuration-mistake case (duplicate server names) that only shows up once you're actually running more than one server.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — Connecting to N servers at once, and almost the whole pool class

Basic Version

```
make an empty routing table, and an empty place to keep each server's session open

for each server in the config list:
    connect to it, do the handshake
    keep its session around (don't close it -- the agent needs it for the whole run)
    list its tools
    for each tool: build the namespaced name, add it to the routing table,
                   add its OpenAI schema to one combined list

give the agent the one combined tools list

when a tool call comes in by namespaced name:
    look up which server it really belongs to
    call that server's session with the real tool name

when the run is completely done: close every connection
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The one genuinely new tool here is `contextlib.AsyncExitStack` — Step 1 and Step 2 only ever had *one* nested `async with stdio_client(...): async with ClientSession(...):` pair, known at write-time. With N servers from a config list, you don't know how many `async with` blocks you need until the program runs, so you can't just nest them by hand in the code.

`AsyncExitStack` lets you enter any number of async context managers at runtime, and closes all of them, correctly, in one `await stack.aclose()` call:

```
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

class MCPServerPool:
    def __init__(self):
        self._sessions = {}
        self._routing = {}
        self._stack = AsyncExitStack()

    async def connect_all(self, servers_config: list[dict]) -> list[dict]:
        openai_tools = []
        for server in servers_config:
            params = StdioServerParameters(command=server["command"], args=server["args"],
            read, write = await self._stack.enter_async_context(stdio_client(params))
            session = await self._stack.enter_async_context(ClientSession(read, write))
            await session.initialize()
            self._sessions[server["name"]] = session

            # your turn: list this server's tools, namespace each one,
            # fill in self._routing, and append its OpenAI schema to
            # openai_tools
        return openai_tools

    async def call_tool(self, namespaced_name: str, arguments: dict) -> str:
        # your turn: look up (server_name, original_name) in self._routing,
        # get that server's session, and call call_mcp_tool on it

    async def close(self):
        await self._stack.aclose()
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in both marked sections, and add the duplicate-server-name check from Hint 1's Advanced section (raise a clear error at `connect_all` time, before any connection is even attempted, if two entries in `servers_config` share a `name`). Also think about `main.py`'s shape now: it needs to call `await pool.connect_all(...)`, run the agent (which now calls `pool.call_tool(...)` instead of a single session directly), and call `await pool.close()` in a `finally` block — so a crash mid-run still closes every subprocess cleanly instead of leaving them running.

Once you've filled it in, compare your finished `MCPServerPool` against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic is the plan in plain language. Intermediate introduces `AsyncExitStack` — the actual mechanism for opening an unknown-at-write-time number of connections and closing them all correctly — with two gaps left for you. Advanced adds the config-mistake guard and the `finally`-block cleanup discipline that makes this safe to actually run more than once without leaking subprocesses.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)